



Research Report

GOPAL P. SARMA, RACHEL STERATORE, SUNNY D. BHATT, GEOFFREY IRVING

Verified Machine Learning Infrastructure

Formal Methods for Trustworthy Artificial Intelligence
Deployment

For more information on this publication, visit www.rand.org/t/RR-A4881-1.

About RAND

RAND is a research organization that develops solutions to public policy challenges to help make communities throughout the world safer and more secure, healthier and more prosperous. RAND is nonprofit, nonpartisan, and committed to the public interest. To learn more about RAND, visit www.rand.org.

Research Integrity

Our research integrity is grounded in RAND's core values of quality and objectivity. Rigorous quality assurance procedures, conflict of interest screening, and transparency in funding ensure that every study is objective and nonpartisan. Learn more at www.rand.org/integrity.

RAND's publications do not necessarily reflect the opinions of its research clients and sponsors.

Published by the RAND Corporation, Santa Monica, Calif.

© 2026 RAND Corporation

RAND® is a registered trademark.

Limited Print and Electronic Distribution Rights

This publication and trademark(s) contained herein are protected by law. This representation of RAND intellectual property is provided for noncommercial use only. Unauthorized posting of this publication online is prohibited; linking directly to its webpage on rand.org is encouraged. Permission is required from RAND to reproduce, or reuse in another form, any of its research products for commercial purposes. For information on reprint and reuse permissions, visit www.rand.org/about/publishing/permissions.

About This Report

Strong feedback loops from automated artificial intelligence (AI) research and development could emerge at frontier laboratories as soon as late 2026, significantly compressing timelines for transformative AI capabilities. If that trajectory materializes, the security of the infrastructure on which these systems run will become an immediate operational concern. In this report, we synthesize findings from structured expert consultations on whether *formal methods*—using mathematical techniques to reason about software behavior and, potentially, to show that systems behave as specified—could meaningfully strengthen machine learning infrastructure before the window of opportunity closes. The consultation was motivated by an assessment that AI capabilities for formal methods might advance rapidly enough to transform the economics of software verification on a similar time frame, creating a narrow window with which to lay the groundwork for verification. The assessment draws on survey responses from 23 experts in domains spanning formal methods, AI infrastructure, cybersecurity, software engineering, hardware architecture, and policy. We examine which components of the machine learning inference and training stacks are most amenable to formal verification, what security properties could be guaranteed, what development and adoption barriers exist, and how advances in AI-assisted formal methods might change the calculus. We offer preliminary recommendations and a starting point for a community-technical roadmap oriented toward frontier AI labs, the formal methods community, hardware vendors, and government agencies, with the intention that these recommendations be refined through rapid iteration as capabilities and priorities evolve.

Center on AI, Security, and Technology

RAND Global and Emerging Risks is a division of RAND that delivers rigorous and objective public policy research on the most consequential challenges to civilization and global security. This work was undertaken by the division's Center on AI, Security, and Technology, which aims to examine the opportunities and risks of rapid technological change, focusing on artificial intelligence, security, and biotechnology. For more information, contact cast@rand.org.

Funding

This research was independently initiated and conducted within the Center on AI, Security, and Technology using income from operations and gifts and grants from philanthropic supporters. The research was completed through funding provided by the Hewlett Foundation. A complete list of donors and funders is available at www.rand.org/CAST. RAND clients, donors, and grantors have no influence over research findings or recommendations.

Acknowledgments

This report went through several iterations, and many people contributed substantially to improving it. We thank several people whose valuable feedback underpins this report: Mauricio Baker, Shivan Bhatt, Alan Chan, Santiago Cuellar, Jordan Despanie, Erich Devendorf, Quinn Dougherty, Anjay Friedman, Kathleen Fisher, Varun Gandhi, James Gimbi, Quentin Hodgson, Emily Lathrop, Alvin Moon, Patrick Roberts, and Mike Vermeer. We also thank the Research Editorial and Production team, including Sean Mintus,

Allison Kerns, and Dan Cabrera, and we thank RAND Global and Emerging Risks's quality assurance team, including Gary Cecchine, Jason Etchegaray, and Paige Smith, for helping us produce and deliver this timely report.

Summary

If frontier artificial intelligence (AI) laboratories begin to see strong feedback loops from automated AI research and development—a possibility that could materialize as soon as late 2026—the software infrastructure running these systems will become the most consequential attack surface in the world. Every AI model behavior, every safety control, and every containment guarantee passes through this infrastructure. Two converging threat models make strengthening the infrastructure’s security urgent: (1) conventional cyber adversaries—nation-states and criminal organizations—targeting AI infrastructure to steal model weights or disrupt operations and (2) loss-of-control threats in which a misaligned AI system exploits vulnerabilities in its own infrastructure to bypass safety monitors or exfiltrate itself. In this report, we present findings from a structured expert consultation on whether *formal methods*—using mathematical techniques to reason about software behavior and, potentially, to show that systems behave as specified—could meaningfully reduce risk from both threat models.

With this report, we aim to identify promising starting points for action; we do not intend for this report to serve as a durable reference. The findings we present are from a structured consultation with 23 experts that we conducted in early 2026—a snapshot of informed judgment at a specific moment in a rapidly evolving landscape. If the scenario motivating this work materializes on the timeline described above, these findings will require revisions within months or possibly sooner. The pace of change is already outrunning the consultation timeline: As we were finalizing this report, Anthropic previewed Claude Mythos, which demonstrated unprecedented AI capabilities in vulnerability discovery, identifying thousands of previously unknown flaws across every major operating system and web browser and underscoring both the severity of the threat landscape we describe here and the narrowing window for establishing verified defenses (Anthropic, 2026; Nolan, 2026). Other frontier labs have since made similar announcements, and it is widely anticipated that open-source models will also match these capabilities within one year. Consequently, the measure of success is whether relevant technical communities convene quickly enough that the priorities identified here are superseded by sharper ones developed through direct collaboration among formal methods researchers, AI-safety practitioners, infrastructure engineers, and policymakers.

A crucial framing note: Formal verification proves that specified components satisfy specified properties under specified assumptions. Formal verification is not, by itself, a security posture. Verification cannot address insider threats, social engineering, or physical side-channel attacks, and its reach into semantic problems (such as model manipulation) is limited. Moreover, the most demanding aspect of verification is not proof generation but *specification*—determining which properties to verify and ensuring that those properties capture the security objectives that matter. This specification challenge, where AI assistance is currently the least reliable and human judgment is the most critical, emerged as a central finding of the consultation and shapes each of our recommendations that follow. Readers should interpret the findings as an identification of how verification can add genuine, bounded value to a broader security strategy—not as a claim that verified infrastructure is secure infrastructure.

Twenty-three experts across the domains of formal methods, AI infrastructure, cybersecurity, and policy responded to a survey to assess risks, barriers, priorities, and opportunities for verification within machine learning (ML) infrastructure. Their responses converge on several key findings:

- **The vulnerability landscape is severe and poses significant risks.** Critical vulnerabilities in major serving frameworks, such as vLLM (CVE-2025-62164, 2025; CVE-2025-9141, 2025) and SGLang (CERT Coordination Center, 2026), along with high-severity vulnerabilities in TensorRT-LLM (NVIDIA,

2025), confirm that ML infrastructure has often prioritized performance and features over security. Experts rated weight integrity and supply chain threats as the most urgent concerns.

- **Verification is effective for infrastructure but not for models.** Formal methods can address code-level correctness—memory safety, access control, protocol enforcement, sandbox isolation—but have limited reach into semantic challenges, such as model sandbagging or jailbreaks. Data poisoning occupies a middle ground: Cryptographic verification can enforce the integrity of data pipelines and detect unauthorized modifications but cannot evaluate whether the data’s content is itself problematic. Most respondents clearly and consistently noted this boundary.
- **Focusing on cryptographic primitives and access control is the clear starting point.** These components combine the highest-security value with the highest-verification feasibility. Production-ready verified implementations already exist, including HACL*/EverCrypt for cryptography—deployed in Firefox, the Linux kernel, and WireGuard VPN (Zinzindohoué et al., 2017; Protzenko et al., 2020; Project Everest, undated; Microsoft Research, 2021)—and Amazon Web Services’ (AWS’s) Cedar for authorization (Cedar, undated).
- **The graphics processing unit (GPU) verification gap is significant but narrowing.** There are no mature full-stack formal semantics for NVIDIA CUDA (the dominant programming model for GPU computation), although partial formal-semantics work exists, and advances in this area suggest that automated GPU kernel verification is becoming more tractable (Chatterjee et al., 2025). In the interim, experts recommended treating GPU computation as a lower-trust component and placing verified wrappers at the CPU-GPU boundary while pursuing formal GPU semantics as a high-priority enabling investment.
- **AI-assisted formal methods are the critical enabler.** There was near-unanimous agreement that AI will dramatically reduce the cost of proof synthesis and repair. Some respondents of our survey projected a two- to five-year horizon for transformative impact, although early adopters have already reported order-of-magnitude productivity gains from AI-assisted proof development. *Specification elicitation*—determining what properties to verify—was the notable exception: Experts warned that AI-generated specifications are vulnerable to subtle flaws that cannot be mechanically checked. A mitigating factor is that many important software transformations share common correctness criteria: A single specification reviewed once can be reused to verify all instances of a transformation class, dramatically reducing the human specification burden.
- **Adoption requires a value proposition beyond security.** In a capability race, frontier labs will not accept meaningful performance or velocity penalties for security purposes alone. Verified infrastructure must be highlighted for its competitive advantages (e.g., fewer bugs, data-breach protection, regulatory positioning).¹ Moreover, if verification eliminates entire vulnerability classes, the relevant Common Vulnerabilities and Exposures (CVE) rate drops dramatically, and the perceived tension with development velocity largely dissolves.

We recommend a phased approach to implementing verified ML infrastructure: Deploy verified cryptographic boundaries immediately using existing technology, then extend to *safety-monitor chains*—infrastructure-level enforcement requiring that model outputs pass through specified filters—and sandbox verification as AI-assisted formal verification tools mature. Subsequently, pursue guarantees of distributed systems correctness as the relevant technologies permit. Making foundational investments in formal meth-

¹ AWS’s Automated Reasoning Group illustrates how verification delivers value well beyond security. The group, in its experience applying formal methods across AWS’s Simple Storage Service (S3) and other services, found that verified code was often more performant than the code it replaced, easier to maintain, and enabled faster release cycles (Cook, 2024).

ods benchmarks (and associated reinforcement learning environments) is a key enabler for this roadmap because it can help create the incentives and training environments needed for AI to dramatically reduce the cost of verification.

These findings should be understood in context. The consultation drew on 23 experts, a convenience sample, not a representative survey, and the findings reflect the perspectives of those specific respondents at a moment when the underlying capabilities and threat landscape are changing rapidly. The roadmap and recommendations in this report should be read as provisional priorities emerging from a structured consultation, not as settled sequencing guidance or a precise deployment plan. The recommendations are intended to identify tractable entry points for further technical refinement—efforts that should begin immediately given the timelines described previously. Many of the verification priorities we identify in this report require further threat modeling to determine which properties matter most and which components belong inside the proof boundary.

Contents

About This Report	iii
Summary	v
Tables	xi
 CHAPTER 1	
Introduction	1
The Security Problem	1
Why Formal Verification	1
Strategic Context.....	2
 CHAPTER 2	
Methodology	5
 CHAPTER 3	
Findings	7
The Risk Landscape.....	7
What Verification Can and Cannot Address.....	8
Component Priorities: Where to Start	9
Barriers to Development and Adoption.....	10
Stakeholder Landscape	12
The Specification Challenge.....	12
AI-Assisted Formal Methods.....	13
 CHAPTER 4	
Toward a Community Technical Roadmap	17
Verified Foundations	17
Verified Control Plane.....	17
Distributed Systems and Crosscutting Challenges	18
Open Questions	18
A Minimal Verified Inference Stack.....	18
 CHAPTER 5	
Preliminary Recommendations	19
For Frontier AI Labs and Hyperscalers	19
For the Formal Methods Community.....	19
For Hardware Vendors	20
For Government Agencies	20
For the Broader Community	20
 CHAPTER 6	
Conclusion	21

APPENDIXES

A. Detailed Methodology.....	23
B. Survey Instrument Summary	27
C. Detailed Survey Data	29
D. Respondent Profile.....	33
Abbreviations.....	35
References	37
About the Authors	41

Available at www.rand.org/t/RRA4881-1

ANNEX

Verified Machine Learning Infrastructure Annex: Survey Instrument

Tables

C.1.	Risk Severity Ratings.....	29
C.2.	Verifiability by Formal Methods	30
C.3.	Component Verification Priorities	31
C.4.	Expected Impact of AI-Assisted Formal Methods in Two to Five Years	31
D.1.	Primary Domain.....	33
D.2.	Years of Experience.....	33
D.3.	Areas of Expertise (Multiple Selections Permitted).....	33

Introduction

The Security Problem

The rapid advancement of frontier artificial intelligence (AI) systems has created an urgent need to secure the infrastructure on which these systems run. This urgency arises from two distinct but converging threat models. The first is the conventional cyber-threat model: nation-state actors, criminal organizations, and other human adversaries targeting AI infrastructure to steal model weights, disrupt operations, or compromise the integrity of AI-generated outputs. The Colonial Pipeline (Parfomak and Jaikaran, 2021) and Change Healthcare attacks (American Hospital Association, 2024) demonstrated that critical infrastructure is already under sustained assault from sophisticated human adversaries; AI infrastructure, representing billions of dollars in training investment and potentially dangerous capabilities, presents an even higher-value target (Nevo et al., 2024).

The second threat model is novel: a highly capable but misaligned AI system exploiting vulnerabilities in its own infrastructure to exfiltrate its weights, bypass safety monitors, or escape containment. Such misalignment could be induced by a human attacker—for example, through fine-tuning or prompt injection—or could emerge from the training process itself. This loss-of-control scenario represents a qualitative shift in the adversary model (Boudreaux et al., 2025): A misaligned model operates from within the infrastructure itself with intimate knowledge of the software stack and a potentially superhuman ability to identify exploitable flaws. The security of machine learning (ML) infrastructure is therefore not merely an information technology concern but a fundamental AI security challenge—one in which both the conventional and novel threat models demand the same response: mathematically verified guarantees about infrastructure behavior.

In mid- to late 2025 and early 2026, security researchers disclosed serious vulnerabilities in multiple major large language model (LLM)–serving frameworks. Researchers found that LLM library vLLM contained flaws that could enable denial of service and potential or direct code execution in exposed serving paths (CVE-2025-62164, 2025; CVE-2025-9141, 2025). SGLang disclosed unsafe-deserialization vulnerabilities that could permit unauthenticated and remote code execution in exposed components (CERT Coordination Center, 2026). Technology company NVIDIA also disclosed a high-severity TensorRT-LLM vulnerability in the Python executor that could lead to code execution, information disclosure, and data tampering (NVIDIA, 2025). Taken together, these cases suggest that portions of the ML-serving ecosystem have accumulated substantial security debt while prioritizing performance and rapid feature development.

Why Formal Verification

Formal verification offers mathematical proofs showing that software behaves exactly as specified, providing guarantees that testing and code review cannot. More broadly, the formal methods on which verification relies—including specification, model checking, and static analysis—can deliver strong assurance even short of full proof. The open-source seL4 verified microkernel (Klein et al., 2009) has had zero functional

correctness bugs in more than 15 years of deployment (seL4, undated). The HACL*/EverCrypt family of verified cryptographic implementations (Zinzindohoué et al., 2017; Protzenko et al., 2020) has been deployed in production systems, including Firefox, the Linux kernel, and WireGuard VPN (Project Everest, undated; Microsoft Research, 2021). The CompCert verified C compiler (Leroy, 2009) guarantees that compiled code preserves the semantics of the source, and IronFleet (Hawblitzel et al., 2015) and Verdi (Wilcox et al., 2015) have demonstrated that formal verification can extend to practical distributed systems. These precedents demonstrate that the verification of systems software is achievable and delivers real security value.

In this report, we examine whether similar techniques could be applied to ML infrastructure. Specifically, we ask the following: Which components of the *inference stack*—the layers of software involved from a user request through a model’s execution and response—and the analogous training infrastructure are most amenable to verification? What security properties could be guaranteed? What are the barriers to development and adoption? How will advances in AI-assisted formal methods change the calculus?

Strategic Context

Verified ML infrastructure sits at the intersection of highly specialized mathematical techniques and urgent geopolitical considerations—a combination that resists tidy organizational frameworks but one that policy-makers and funders must navigate as a whole. What follows is deliberately presented as a cluster of strategic intuitions rather than a single thesis: The underlying AI capabilities are shifting quickly, the considerations are diverse because the problem is diverse, and we present the capabilities together because decisionmakers must weigh them together. Nonetheless, one factor stands apart: If AI-assisted formal methods improve rapidly, the economics of verification might change rapidly enough to justify immediate investments in verification-relevant infrastructure, especially benchmarks, specifications, and tractable deployment targets.

The consultation we conducted was part of a broader strategy examining high-assurance systems for advanced AI in preparation for the possibility that transformative AI capabilities, including automated AI research and development, might emerge within the next several years (Sarma et al., 2026). A central premise of this strategy is that AI capabilities for formal methods and mathematical reasoning are likely to advance disproportionately relative to other AI capabilities. The key structural reason is that proof assistants provide a near-perfect reward signal,¹ making formal verification an ideal target for reinforcement learning—in contrast to most software-engineering tasks, for which correctness evaluations are approximate (Sarma and Fisher, 2026; Yang et al., 2025).

A series of rapid advances in auto-formalization and related techniques served as a key motivating example for this effort. Between mid-2025 and early 2026, Math, Inc.’s Gauss agent progressed from completing a formalization of the strong prime number theorem in three weeks—a project that had stalled after 18 months of effort from human experts (Math, Inc., undated-b)—to completing a 200,000-line formalization of Maryna Viazovska’s Fields Medal-winning sphere-packing proof (Math, Inc., undated-a). Saraswat et al. (2026) demonstrated that using AI-assisted proof generation was 350 times faster than a manual verification effort, and the authors’ analysis of a time-horizon framework developed by the AI research organization METR suggests

¹ We say “near-perfect” rather than “perfect” for three reasons. First, although a completed proof provides an unambiguous correctness signal, the reward can be very sparse in practice: Long proofs require intermediate definitions and lemma formulations for which no automated reward is available, making the search problem considerably harder than the binary “check” or “no-check” framing suggests. Second, proof checking itself is not instantaneous—verification attempts can time out, yielding no signal at all rather than a definitive “accept” or “reject” result. Third, the reward signal is only as reliable as the proof checker itself; the presence of any bugs in the checker’s trusted kernel means that a “verified” proof might not correspond to actual correctness. In practice, mature proof assistants have very small, trusted computing bases, which mitigates but does not eliminate this risk.

that AI capabilities for verified software engineering are closing the gap with unverified software engineering faster than expected (see Kwa et al., 2025; for a broader survey of these developments, see Avigad, 2026).

Although several of these results primarily concern pure mathematics, the underlying capabilities are likely to transfer to software verification. Targeted investments in evaluations and reinforcement learning environments that are optimized for software verification tasks will be needed to realize this potential (Sarma and Fisher, 2026). If this trajectory continues, the economics of verification, which have historically been prohibitive because of the cost of using human proof engineers, could shift dramatically within the next one to two years (as of early 2026). As Sarma and Fisher (2026) argue, rigorous benchmarks for AI-assisted formal methods would reshape the competitive landscape:² “If the market comes to value security benchmarks the way it now values math and coding tests, competitive pressure will drive AI labs to invest in verifiable safety.” Early efforts with such benchmarks include VeriBench (Miranda et al., 2025), research from Bursuc et al. (2025), and VERINA (Ye et al., 2025). Without such benchmarks, meaningful progress is still possible and should begin immediately, but progress will remain constrained by the limited pool of human experts in formal methods. With rigorous benchmarks, however, AI systems could dramatically accelerate defensive preparedness by generating verified code, proving security properties, and systematically hardening infrastructure at a pace that manual effort cannot match.

This urgency is compounded by a competitive dynamic: Chinese labs have made rapid advances in integrating theorem proving with AI—notably DeepSeek-Prover-V2 (Ren et al., 2025), which achieved state-of-the-art results on major benchmarks. U.S. frontier labs, meanwhile, have invested comparatively less in this specific capability area, although Google DeepMind’s AlphaProof (Google DeepMind, 2024) and related efforts represent important exceptions (for a broader analysis of competitive dynamics, see Pavel et al., 2025).

An academic AI and ML software engineer, who was one of the 23 experts we consulted, characterized the existing industry posture starkly: The default plan is to “come in with a hilariously low level of security and pray that AIs that fully automate software engineering at very high speeds can achieve nation-state-proof security in a short period of time.” Yet this plan contains its own contradiction: According to the same engineer, “We won’t trust these AIs, so having them rewrite everything may well be catastrophic and will be massively rushed.” This framing underscores the urgency of investing in verification infrastructure and benchmarks now rather than relying on a future in which trusted AI systems can retroactively secure what was built without assurance. Importantly, the institutional foundation for such investment already exists: The Defense Advanced Research Projects Agency’s (DARPA’s) Information Innovation Office has built more than a decade’s worth of momentum in formal methods by developing programs spanning verified military systems (High-Assurance Cyber Military Systems, or HACMS), proof engineering for nonexpert developers (Pipelined Reasoning of Verifiers Enabling Robust Systems, or PROVERS), verified legacy-code modernization (Verified Security and Performance Enhancement of Large Legacy Software, or V-SPELLS), an automated C-to-Rust translator (Translating All C to Rust, or TRACTOR), and AI-assisted mathematical reasoning (Exponentiating Mathematics, or expMath) (DARPA undated-b; DARPA undated-a).

As we were finalizing this report, Anthropic’s Claude Mythos preview demonstrated unprecedented AI capabilities in vulnerability discovery, identifying thousands of previously unknown flaws across every major operating system and web browser. Other frontier labs have since announced comparable capabilities and adopted similar restricted-access programs, suggesting that these advances reflect a broad trajectory in the field rather than an idiosyncratic result. These episodes reinforce two central themes of this report. First, AI capabilities that are relevant to software security are advancing faster than anticipated; Anthropic referred

² Throughout this report, we use *benchmarks* as a shorthand for the broader set of technical artifacts needed to improve AI performance on verification tasks—including standardized evaluations, reinforcement learning environments, training datasets, and other infrastructure that provides signals from which AI systems can learn.

to this advancement as a “step change” rather than incremental progress (Nolan, 2026). Anthropic restricted Mythos to a select group of partners for defensive security work rather than releasing it publicly, citing concerns that attackers could weaponize the model’s capabilities (Anthropic, 2026). Second, the window for establishing verified infrastructure is narrowing—if models of Mythos’s caliber can find vulnerabilities at scale, the case for mathematically verified defenses becomes more urgent than a long-term aspiration. The episodes also reinforce the centrality of the benchmarks and reinforcement learning environments we discuss throughout this report: These models’ capabilities emerged from general improvements in coding and reasoning, which is precisely the trajectory that makes formal verification tasks increasingly tractable for AI systems.

Methodology

Our findings are based on structured survey responses from 23 subject-matter experts across various domains. We developed the survey instrument through an iterative process that included generating simulated expert responses from LLM-constructed personas, which helped refine our question framing and identify gaps in coverage before administering the survey to human respondents. This human-AI collaborative approach informed not only the development of our survey instrument but also the broader process of developing and writing this report; we describe the full methodology in detail in Sarma et al. (2026). We distributed the survey to experts whom we found through our professional networks and by using targeted outreach; all findings, quantitative ratings, and quotations in this report derive from the responses of the human experts we consulted.

The 23 experts completed the survey, and 22 of the experts provided background information. Respondents' primary roles included researchers and academics (45 percent), practitioners and engineers (23 percent), and roles that were of a hybrid character (32 percent). The majority of the respondents reported 11 to 20 years of relevant experience (55 percent); the remainder were split between having six to ten years of experience (32 percent) and zero to five years of experience (14 percent). Areas of expertise (some respondents had more than one area of expertise) spanned software engineering (50 percent), formal methods and verification (50 percent), AI infrastructure and ML systems (50 percent), cybersecurity (36 percent), policy and governance (18 percent), and hardware and systems architecture (5 percent). One coauthor of this report (Geoffrey Irving) also participated as a survey respondent; we included his responses in the analysis without giving them special weighting.

The survey instrument covered six major areas. The first area, security risk assessments, involved the structured ratings of 12 predefined risk categories.¹ The remaining five areas were verification feasibility and adoption barriers, component prioritization (which involved giving security value and feasibility ratings for 15 infrastructure components), specification challenges, AI-assisted formal methods, and related work. All quantitative ratings were accompanied by open-ended explanations, which formed the primary basis for the qualitative findings that we report here. We synthesized the open-ended responses by using an LLM-assisted qualitative analysis that was independently validated by a member of our team trained in qualitative research design and analysis. The team member reviewed the responses and identified key themes, and we incorporated findings from this independent review into this report. Full methodological details and detailed data tables appear in Appendixes A through D. An annex to this report containing the complete survey instrument is available at www.rand.org/t/RA4881-1.

¹ Throughout this report, we use *threat* and *risk* in their broadly accepted meanings: *threat* refers to potential sources of harm or adversary capabilities, and *risk* encompasses the likelihood and impact of those threats being realized. We do not maintain a strict terminological distinction between the terms, which is common in security policy literature.

Findings

The Risk Landscape

The risk landscape that we present here reflects expert assessments of severity and verifiability; it is not a formal adversary model. Developing a rigorous threat model that specifies adversaries, trusted computing bases, and attack mechanisms for each risk is a necessary next step, and we intend for our report to inform that step. For each proposed verification target, the relevant adversary model, trusted computing base, and proof boundary should be specified explicitly. In particular, claims about access control, bypassing monitors, sandboxing, and distributed execution depend on which components are trusted, which attack paths are in scope, and which properties are being verified.

For this survey, we asked experts to rate the severity of 12 predefined risks on a four-point scale (“low,” “medium,” “high,” or “critical”) and to identify additional risks not listed.

One respondent noted that the predefined categories mixed attack objectives (e.g., weight exfiltration) with attack vectors (e.g., remote code execution via serving framework vulnerabilities) and that separating these dimensions would enable a more precise risk analysis. We retain the survey’s original framing here for consistency (along with the original survey instrument in the separate annex), but we note that this was a limitation; future iterations of similar surveys would benefit from distinguishing attack objectives from the vectors through which they are achieved.

Three risk categories received the highest mean severity ratings. Those categories are as follows:

- **Weight integrity** was the dominant concern. Tampering with stored weights or model files received the highest mean severity rating: Eleven of 21 respondents rated the category as critical,¹ and seven rated it as high. The concern of weight exfiltration followed closely; 52.4 percent of respondents rated it as critical. Together, these categories reflect the respondents’ central fear that model weights, representing billions of dollars in training investment, are inadequately protected by current infrastructure.
- **Supply chain compromise** was also among the top-rated risks, with 12 high and seven critical ratings out of 21. The Log4Shell vulnerability (Cybersecurity and Infrastructure Security Agency, 2022) and the xz backdoor attempt (Freund, 2024) have demonstrated that a single compromised dependency can affect thousands of organizations. Multiple respondents emphasized that the ML stack involves tens of millions of lines of code from dozens of sources written in languages that are largely not memory-safe.
- **Remote code execution** via serving framework vulnerabilities also received high and critical risk ratings (it received seven critical ratings and nine high ratings), reflecting an awareness of the recent wave of Common Vulnerabilities and Exposures (CVEs) in vLLM and comparable frameworks.

Beyond the 12 predefined categories, respondents surfaced additional risk categories not included in the original survey instrument. Several respondents emphasized loss-of-control risks: timing attacks against AI

¹ Two respondents did not complete the risk severity ratings; totals in this section reflect 21 responses.

control algorithms, rogue deployments (in which models bypass monitors while consuming inference tokens at scale), and the sabotage of safety experiments. Such scenarios might manifest as a large volume of simultaneous model actions across complex, distributed deployments, overwhelming human audit queues. Other risk categories that respondents identified were physical and human-layer threats—such as data center infiltration, social engineering, and coercion—that formal verification cannot address but that must be part of any comprehensive security plan.

Beyond these, respondents identified additional risk categories spanning four domains. Operational risks included denial of service, stability failures from concurrent operations, performance sabotage, and inadequate detection visibility or response capabilities. Respondents rated several of these categories as high or critical in severity. Data and privacy risks included training-data enumeration, data leaks of protected intellectual property or customer data, retention of user data or metadata meant to be erased, theft of synthetic-training data repositories, and mislabeled data or incorrect ontology. Model behavior risks extended beyond sandbagging to include model-capability extraction through distillation and training and test data leakage.

Respondents also identified a category of risks specific to the verification process itself: safety-invariant violations, deadlocks and data races, and the misuse of verification escape-hatch mechanisms (such as “sorry” in Lean, “Admit” in Coq/Rocq, or “assume” in Dafny) that allow unproven assumptions to be treated as established facts. One respondent warned that specification errors are fundamentally impossible to verify—that is, a specification that fails to capture the intended security property cannot be detected by the proof system itself. As AI-generated proofs become more prevalent, these verification-specific risks take on added significance: An AI system could exploit escape hatches or derive valid proofs from flawed specifications in ways that are difficult for human reviewers to detect.

The verifiability of these additional risks varied (respondents rated the verifiability on a scale of “yes, completely,” “yes, partially,” “no,” and “don’t know”). Respondents considered operational risks, such as stability failures and excessive infrastructure loads, to be fully addressable through verification, while denial of service was rated as only partially addressable. Most data and privacy risks fell outside the scope of infrastructure verification: Training data enumeration and data retention are semantic or policy problems rather than code correctness issues, although verification can help ensure that access controls and data-handling pathways function as specified. Verification escape-hatch risks are addressable through tooling and process controls (e.g., the automated detection of admitted axioms), while specification errors require human review, red-teaming, and the multistakeholder validation processes (which we discuss later in the section titled “The Specification Challenge”).

What Verification Can and Cannot Address

A central finding from our survey is that there is a consistent, clearly articulated boundary around what formal verification can achieve. Experts converged on four domains in which verification faces fundamental limitations:

- **The human boundary.** Social engineering, insider threats, and physical access attacks cannot be addressed through formal methods. As one respondent noted, “Humans must be able to access models. A major threat vector is insider threats from humans with correct credentials. Formal methods cannot solve this problem.”
- **The physics boundary.** Physical side-channel attacks (e.g., involving power consumption or electromagnetic interference) require physical defenses, not code proofs. Timing side channels are a partial exception: With machine-code specifications and verified compilers, it is possible to prove that code executes in constant time, although this task remains beyond the current reach of most ML infra-

structure. Experts noted that side-channel properties are often *hyperproperties*—properties of multiple execution runs—that are notoriously difficult to establish in concurrent systems.

- **The semantic boundary.** Training-data poisoning, model sandbagging, and jailbreak attacks involve the meaning of data and model behavior, not the correctness of code. Several experts emphasized that formal methods can secure data “at the bit level, not the semantic level”; that is, formal methods can detect unauthorized modifications but not evaluate whether the data’s content is problematic.
- **The modeling boundary.** Verifying system interactions (e.g., file inputs and outputs, system calls [syscalls], memory models) requires accurate formal models of an operating system’s (OS’s) behavior—and the quality of the guarantee is only as good as the model it is built on. One respondent illustrated the risk of incomplete models concretely: “We verify that the model does not write to its own stored weights by monitoring the fs write() syscall. But we forgot to mark some other syscalls that also can be used to write files, or [we] failed to account for symlinks [symbolic links].” The lesson is not that verification is fragile, but that it depends on competent system modeling and clear assumptions about what the proof does and does not cover. Multiple respondents recommended that verifying system interactions should be combined with runtime sandboxing and isolation rather than be relied on as a stand-alone guarantee. Existing verified foundations, such as seL4, can help narrow this gap by providing a formally specified OS layer, but even verified OS kernels make modeling assumptions that might not capture all real-world system behavior.

Within its proper domain, however, respondents viewed verification as highly effective. Remote code execution vulnerabilities, unsafe deserialization, and inadequate tenant isolation received the most “yes, completely” verifiability ratings (ten to 11 out of 22 respondents each). The vulnerabilities described previously are quintessential code-correctness problems: memory safety, input validation, and access control enforcement in which specifications are well-defined and tools are mature.

The majority of risks fell into a middle ground that respondents rated as partially verifiable. Weight exfiltration, for example, received 14 “yes, partially” ratings. The respondents explained that formal methods can prove that network paths to the outside world run specified cryptographic protocols but cannot prevent exfiltration through physical access or social engineering. As one respondent put it, verification might cover “10 or 15 percent of the attack surface” for weight exfiltration—a modest fraction, but one that eliminates an entire class of software-based attacks.

Component Priorities: Where to Start

Experts rated 15 ML infrastructure components on two dimensions: how much security value verification would provide (using a five-point scale from “minimal” to “very high”) and how feasible verification is with current tools (using the same five-point scale). The results show a clear starting point and a logical sequence for expanding the scope of verification. We categorized the results into three tiers, as follows:

- **Tier 1—Immediate targets.** Cryptographic primitives, access control and authentication, and sandbox and isolation boundaries each received mean security value ratings near 3.8 out of 5 (a rating of high). Cryptographic primitives also received high feasibility ratings (with a mean of 4.5), which is unsurprising given that production-ready verified implementations already exist. Access control had a mean score of 4.2 on feasibility, reflecting mature tools, such as Cedar’s verified authorization framework (Cedar, undated). These components offer the highest return on verification investment.
- **Tier 2—High value, moderate difficulty.** Communication protocols (e.g., Transport Layer Security [TLS], a remote procedure call [RPC]) had a mean score of 3.6 for its security value and a feasibility

score above the midpoint, which is likely because communication protocols have benefited from existing verified-protocol work (Blanchet, 2001; Meier et al., 2013). The OS and hypervisor layer received a security value mean score of 3.8 but a lower feasibility mean score of 2.7, although experts noted that seL4 already provides a verified foundation. Hardware drivers had a mean security value score of 3.3 but received low feasibility ratings because their verification requires modeling proprietary timing requirements that manufacturers guard closely. There was notable disagreement when rating *safety monitors* (the infrastructure-level enforcement of passing all model outputs through specified filters—including audit logging, output mediation, and runtime gating—before reaching users). Although several respondents supported verifying that all outputs pass through specified filters before reaching users, others argued that the monitors themselves are ML-side defenses, and bypassing the monitors is a semantic rather than infrastructure problem. Therefore, verification would not yield tangible security benefits. The less contested formulation focused narrowly on verifying the monitor *chain*—that is, ensuring that monitors cannot be skipped or circumvented rather than verifying monitor *effectiveness*, which depends on ML capabilities outside the scope of infrastructure verification.

- **Tier 3—Important but challenging.** Graphics processing unit (GPU) kernels and the ML framework core both received moderate security value ratings (3.0 and 2.9, respectively) but very low feasibility scores (2.2 and 2.1, respectively). Although formal semantics and verification work exists for parts of NVIDIA’s CUDA platform’s parallel thread execution (PTX) stack—including executable semantics for heterogeneous CUDA-C, formal analysis of the NVIDIA PTX memory model, and formal verification tools for CUDA kernels—there is still no mature, widely adopted full-stack formal semantics for modern CUDA that is sufficient for end-to-end verification of production GPU kernels (Hathhorn et al., 2012; Lustig, Sahasrabudhe, and Giroux, 2019; Betts et al., 2015; LLVM Project, undated). The LLVM Project’s Clang compiler provides an upstream alternative compilation path for CUDA, and research suggests that automated verification of CUDA kernels is becoming more tractable (LLVM Project, undated; Chatterjee et al., 2025). Multiple experts, therefore, recommended treating GPU computation as a lower-trust component, with verified wrappers enforcing input validation, output integrity, and access control at the CPU-GPU boundary.

Several respondents highlighted an architectural implication of these priorities: The most effective strategy is not to verify everything but to separate verified, slowly changing security-enforcement components from unverified, rapidly evolving optimization code, putting type-system-level guarantees at the interface.

Barriers to Development and Adoption

The barriers we identify in this section include verification-specific challenges (such as the specification burden and the absence of formal tools for Python) and broader platform transition costs (such as switching costs and proprietary dependencies) that would accompany any major infrastructure change. We include both challenges because respondents raised them together, but the distinction matters: The verification-specific challenges are the barriers in which targeted investments can make the most difference.

Technical Barriers

The most frequently cited technical barriers were scale and heterogeneity: ML infrastructure involves millions of lines of code across Python, Rust, C++, and CUDA, and components come from dozens of sources. Much of ML infrastructure runs in interpreted languages whose behavior is undefined until execution. In particular, multiple respondents noted the absence of formal verification tools for Python, the dominant lan-

guage in ML infrastructure, as a concrete and immediate gap. The ML stack also depends heavily on proprietary components (NVIDIA’s CUDA runtime, Google’s libtpu) whose internals lack any formal specification. However, verification does not need to extend to proprietary internals: it is possible to write formal specifications for the interface, assume interface correctness for verification purposes, and gain confidence in those assumptions through extensive testing.

Respondents cited the specification challenge nearly as often as they cited the technical barriers. Multiple experts emphasized that defining what “correct” means for a given component consumes 30 to 50 percent of verification efforts. Floating-point non-associativity, concurrency, and distributed states all resist simple specification. One respondent characterized the core difficulty: “Anything that can’t be verified must be specified; any mismatch in this spec can cause bugs.”

Adoption Barriers

Performance and velocity concerns dominated the adoption discussion. Every frontier lab is locked in a capability race; respondents noted that even a 10-percent performance penalty was potentially disqualifying, a concern that is consistent with the competitive dynamics between countries in the race to achieve artificial general intelligence (as analyzed by Mitch et al. [2025] and Abraham, Kavner, and Moon [2026]). This constraint reflects the competitive dynamics of frontier AI development specifically; in high-assurance contexts, such as with defense or safety-critical systems, substantial performance penalties are routinely accepted in exchange for stronger security guarantees. The speed of development is an equally significant barrier: Formally verified code is widely perceived as harder to modify, and proof repair could slow iteration cycles that run on weekly cadences—although, as noted previously, AWS’s experience suggests that verification can ultimately accelerate rather than impede development velocity (Cook, 2024).

Transition costs were the second major barrier to adoption. Frontier labs have invested heavily in custom inference stacks with bespoke internal tooling. The switching costs of moving to verified infrastructure are enormous and compounded by the fact that labs treat their stacks as proprietary. One respondent captured the trust problem: “If someone told me right now [that] they had a ‘verified ML infrastructure stack,’ I would simply assume [that] they did not know what they were talking about.” Respondents also described a “bugs-are-features” problem: Workarounds compensating for bugs in legacy code become load bearing over time so that fixing one bug during a transition to verified infrastructure can break the system in larger and less predictable ways.

Several respondents argued that adoption will not happen using security arguments alone. One respondent proposed reframing verification as a process that enables a competitive advantage: “Model weight security is *not* a USP [unique selling proposition]. Data security is. Apple has shown that investing in systems that lock its own engineers out of user data creates a USP even for consumers.”

Confidentiality constraints present an additional obstacle. Multiple respondents noted that frontier labs treat their inference stacks as proprietary for commercial, security-via-obscurity, and antiproliferation reasons. A verification approach that requires sharing stack details or adopting common infrastructure might directly conflict with these constraints. This suggests that verification tools, specification libraries, and proof methodologies (as opposed to verified stacks themselves) might be the appropriate ways to share information across organizations. This distinction has direct implications for the roadmap: Investment should prioritize portable verification tooling that labs can apply to their own proprietary stacks rather than assuming convergence on a single shared verified infrastructure.

At the individual developer level, there is an additional friction to adoption: Developers would need to write or engage with specifications, and many developers are unwilling or unable to do so, or they simply do not know what the correct specification should be. This is a distinct barrier from the technical difficulty of specification that we discuss in the next section and reflects a practical adoption barrier at the point of use.

The Patch Velocity Debate

In reviewing the survey findings, a tension emerged regarding the compatibility of formal verification with the need to rapidly evolve production systems: whether to patch vulnerabilities, ship new features, or adapt to changing infrastructure. The economic and competitive challenges mentioned previously are compounded by the fact that much of the infrastructure that verified systems would need to compete with, such as Linux-based hypervisors, is available at no cost. Respondents noted that formal methods are widely perceived as “a cost without a clean value proposition,” making it difficult to secure funding for verified alternatives that must compete on both performance and price with mature, free, open-source infrastructure.

Several respondents took the pragmatic view that verified components should be layered over strong test coverage, and temporary deviations from formalization should be permitted when urgent patches are needed. Several experts proposed architectural solutions: using modular designs in which verified security-enforcement code changes infrequently while unverified optimization code iterates freely and is connected through type-system guarantees (following the model of AWS Cedar’s verified sidecar architecture).

A sharply contrasting view challenged the premise entirely. A software engineer with expertise in formal methods and verification argued that this concern “is, frankly, ridiculous.” The respondent pointed out that seL4’s fully verified configurations have never had a functional correctness bug (Klein et al., 2009; seL4, undated), and later seL4 releases have disclosed critical security-relevant defects affecting only unverified configurations (seL4, 2025). These facts weaken the alternative explanation that the verified configuration’s clean record is simply because of insufficient attacker interest. The implication is direct: If verification removes whole classes of bugs before deployment, the need for urgent patches should fall substantially.

Stakeholder Landscape

Respondents identified several stakeholder groups whose involvement would be necessary for the development and adoption of a verified ML infrastructure stack. Frontier AI labs are the most critical stakeholder: Lab leadership needs to be convinced that verification will not compromise performance and that no cheaper fixes exist, and infrastructure engineers and ML developers need assurance that verification will not limit flexibility and features. Chief compute officers could be persuaded of verification’s value if verified stacks demonstrably improve compute efficiency. Respondents emphasized that each lab would need to actively participate in creating stacks that work for their specific infrastructure; a one-size-fits-all approach will fail.

Hardware manufacturers, particularly NVIDIA, are essential for providing formal instruction set architecture semantics and verification-friendly chip designs. Hyperscalers and cloud service providers would benefit from security guarantees but would also require performance parity. Respondents identified governments as being necessary stakeholders so that they could apply regulatory pressure and fund shared development, especially because competitive dynamics alone would be unlikely to drive adoption. Open-source community influencers (e.g., maintainers of vLLM, Triton, RayServe, Unsloth) would need to endorse the effort for credibility within the ML engineering community.

A crosscutting theme we surfaced from the survey responses was that trusted voices from multiple domains—including domains of formal methods, AI safety, infrastructure engineering, and industry leadership—must endorse the effort for it to achieve the broad credibility needed for adoption.

The Specification Challenge

Respondents offered detailed thinking on what specifications should capture and how the specifications should be structured. **A recurring principle in responses was that specifications should operate at a higher**

level of abstraction than naïve formulations suggest. As an example, rather than specifying that “only holders of key K can decrypt weights” (which fails if the key leaks), a formal methods and verification software engineer argued for specifications that could read similarly to the following: “No information about weights is leaked except to nodes possessing a currently valid verifiable credential running a container authenticated by the current Authorization Policy for Employee Weight Access.” This specification illustrates both the power and the limits of higher-level abstractions: It captures a richer security property than naïve formulations, but it also encodes trust assumptions—about who issues credentials, who authenticates containers, and which authorization policy is trusted—that sit outside the proof boundary. If those assumptions are wrong, the verified component can be correct while the system remains insecure.

Several experts proposed an organizing principle for specifications particular to ML: Prove that high-performance distributed code is equivalent to simple reference implementations expressed in mathematical domain-specific languages. Under this approach, a neural network architecture would be defined in a simple tensor language without batching or single instruction, multiple data (SIMD); the mathematical model would be allowed to be ambiguous (e.g., permitting any association of floating-point operations); and the verification goal would be to show that the production system’s behavior falls within the set of behaviors permitted by the mathematical model.

This approach also illustrates how floating-point non-associativity (often cited as a barrier to verification) can be addressed through creative specification rather than excluded as intractable. Rather than assuming that floating-point arithmetic is associative (which would be unsound), it can be possible to verify that (1) a GPU kernel faithfully evaluates some circuit within the same associativity-equivalence class or that (2) computed results fall within a bounded error of a reference implementation. These are incomplete but sound specifications: They do not capture every property of interest, but the properties they do capture are genuinely proven. Respondents noted that similar approximate specification strategies (e.g., equivalence-class approaches, bounded-error tolerances, and ambiguous mathematical models) represent active areas with practical promise for ML infrastructure, provided that such approximations cannot be “stacked” to produce compounding errors through a pipeline.

A critical distinction emerged between incomplete and unsound specifications. Incomplete specifications—those that verify some, but not all, desired properties—were widely seen as valuable, providing meaningful risk reduction even when they do not capture every threat. Unsound specifications—those that assume known-false conditions—were identified as dangerous, particularly in the context of AI-generated proofs. As one formal methods and verification software engineer warned, “As we grow to rely on AIs to generate proofs, unsound specifications will become worthless. AIs will be able to derive any desired outcome from a false assumption, and they will likely be able to sneak the fishy reasoning past us.” Respondents also cautioned that incomplete specifications carry a compounding risk: Small approximations that are acceptable individually can become dangerous if they stack through a pipeline. For example, bounded floating-point errors at each stage of computation might accumulate into large errors across an end-to-end system. Specification strategies should account for how approximations compose, not only whether each individual approximation is tolerable.

AI-Assisted Formal Methods

Every respondent identified AI-assisted formal methods as the most important variable determining whether verified ML infrastructure is achievable at reasonable costs. The survey asked respondents to rate how AI advances over the next two to five years (that is, 2028 through 2031) would affect the feasibility and cost of four verification tasks.

There was near-unanimous optimism from respondents about **proof synthesis and repair**. Seventeen of 22 respondents expected proof repair to become “significantly easier,” and 17 expected “significantly lower” costs. **Proof synthesis** showed similar patterns (16 respondents said it would become “significantly easier,” and 16 said costs would be “significantly lower”). These ratings reflect the view that proof generation and maintenance are bottlenecked by labor constraints, not by the fundamental difficulty of achieving them, and are therefore highly amenable to AI assistance.

Specification elicitation was the clear outlier. Although a plurality of respondents expected improvement in specification elicitation, five respondents expected there to be “no change” in the feasibility of enacting it, and one expected feasibility to become “significantly harder.” Specification elicitation was the only task in which any respondent predicted regression. The concern was not that AI cannot help with specification—it can—but that specification is uniquely vulnerable to human-reward bias. An AI systems engineer explained: “Specification elicitation is the [capability] that seems most vulnerable to human-reward bias, where the AI produces a compelling but subtly flawed answer . . . Proof synthesis and repair don’t have this vulnerability because their outputs can be verified outright.”

One promising approach to this specification bottleneck is the development of *task-level specification generators*, which are tools that define correctness one time for an entire class of software operations and then automatically generate instance-specific specifications for any codebase. Saraswat et al. (2026) demonstrates this approach for verified translation between proof assistants, achieving a 350-fold increase in speed compared with manual verification efforts by reusing a single specification across more than 1,200 theorem statements. Because many critical software operations (e.g., translation, optimization, refactoring, parallelization) are semantics-preserving transformations, a single human-reviewed specification can cover arbitrarily many instances, reducing the oversight burden from $O(n)$ to $O(1)$. Additionally, in domains where candidate specifications can be validated empirically—through fuzzing, differential testing, or making comparisons against existing informal documentation—AI systems can accelerate specification generation directly, mitigating the human-reward bias concern by grounding AI-generated specifications in observable behavior. This suggests that the specification bottleneck, while real, might be more tractable than individual-component reasoning would imply.

This asymmetry carries a strategic implication: As AI makes proofs cheaper, the relative bottleneck shifts to specification, which is precisely the task at which AI assistance is presently the least reliable and human judgment is the most critical. Investments in developing specification methodologies—including red-teaming specifications, developing shared specification libraries, and building tools for specification validation—will become increasingly important. This also means that the phased roadmap that we propose in this report should be understood as being contingent on progress made in developing specification methodologies, not only on advances in proof automation.

Several respondents noted a deeper uncertainty: Even if AI dramatically reduces the absolute cost of producing verified code, AI will simultaneously reduce the cost of producing unverified code. The relative cost advantage of verification might therefore not improve and could even widen if the production of unverified code benefits more from AI assistance (for example, because running unverified code does not require the structured reasoning that proof generation demands). This carries a further implication: Even under optimistic assumptions about AI-assisted formal methods, adoption might still require external pressure—whether through regulation, liability frameworks, insurance requirements, or procurement mandates—to shift the cost-benefit calculus in favor of verification. The AI-assisted formal methods trajectory described previously is a necessary but potentially insufficient condition for widespread adoption.

This complex reality makes the development of comprehensive *formal methods benchmarks*—evaluations measuring AI performance on specification writing, theorem proving, verified code generation, and memory-safe translation—an especially important enabling investment, although the priority of develop-

ing such benchmarks relative to other interventions depends on the broader strategic context and should be revisited as the field evolves. As Sarma and Fisher (2026) argue, public leaderboards for formal methods performance could create competitive pressure analogous to existing math and coding benchmarks, driving frontier labs to develop capabilities that receive little attention. Benchmark development should begin immediately, and initial suites should target proof repair (the highest-consensus near-term AI application that we identified from the survey responses), memory-safe code translation, and systems-level reasoning about memory, system state, and concurrency.

Toward a Community Technical Roadmap

What follows in this chapter is not a definitive roadmap but a starting point for community deliberation. We present it to structure the conversation that should follow this report, ideally within weeks, not months, and expect it to be revised substantially through direct engagement with the stakeholders identified previously.

The workstreams we mention are largely independent of one another and can proceed in parallel; we order them roughly by the maturity of existing tools, but community priorities and emerging capabilities should dictate how to concentrate effort first. Our goal is not to prescribe a sequence but to identify tractable starting points that can galvanize work across the formal verification and AI safety communities. Each workstream’s feasibility depends not only on proof automation but on the availability of sound, reviewable specifications—a dependency that we discussed in Chapter 3 that should inform how aggressively any individual workstream is pursued.

Verified Foundations

Stakeholders should

- deploy verified cryptographic libraries for weight protection (Zinzindohoué et al., 2017; Protzenko et al., 2020; Nevo et al., 2024)
- develop formal specifications for access control and authentication in inference systems, building on existing frameworks, such as Cedar
- engage hardware vendors on accelerator instruction set architecture semantics to expand the verifiable surface
- assemble formal methods benchmarks (Sarma and Fisher, 2026) to drive AI capability development on verification tasks
- explore open-source hardware (e.g., RISC-V accelerators) as a longer-term path to building verifiable AI compute.

Verified Control Plane

Stakeholders should

- verify safety monitor chains (as noted in Chapter 3, the value of this workstream lies in verifying that monitors cannot be bypassed or circumvented at the infrastructure level; the effectiveness of the monitors themselves is an ML problem that infrastructure verification cannot address, and this distinction should guide specification efforts)
- verify sandbox and isolation boundaries for tool use and code execution, building on seL4-style capability systems

- architect the separation of verified security-enforcement modules from unverified performance-optimized code, and include type-system guarantees at the interface to ensure that unverified components cannot bypass the security invariants enforced by verified components
- leverage AI-assisted proof synthesis as capabilities mature.

Distributed Systems and Crosscutting Challenges

Stakeholders should

- verify that distributed coordination protocols for tensor and pipeline parallelism produce results equivalent to single-machine execution, including under *checkpoint swaps*—applying and extending existing distributed verification frameworks (e.g., IronFleet, Verdi) when ML-specific parallelism patterns require them
- verify training orchestration integrity when feasible
- develop specification methodologies—including red-teaming, multistakeholder review, and automated consistency checks—that mitigate the risk of subtle flaws in AI-generated specifications.

Open Questions

We exclude ML model behavior (alignment, sandbagging, jailbreaks) from this roadmap because the behavior requires interpretability advances, not infrastructure verification. However, work on formalizing neural network semantics directly within theorem provers (George et al., 2026) suggests that the boundary between model verification and infrastructure verification might narrow as these tools mature. GPU kernel verification, while not yet feasible with existing tools, is a near-term goal rather than a fundamental limitation.

A Minimal Verified Inference Stack

In parallel with using component-level verification, a tractable proof of concept would be to target a narrowly scoped verified inference pipeline—such as one with tens of thousands of lines of verified C and CUDA code—with explicitly defined security properties, a bounded trusted computing base, and a clear threat model. Such projects as Andrej Karpathy’s `llm.c` and `llama2.c` (Karpathy, 2024; Karpathy, 2025) demonstrate that LLM training and inference can be written in a few thousand lines of plain C and CUDA code once the model architecture is established, making holistic verification substantially more tractable than the scale of production ML frameworks might suggest. Such an effort would complement the piecemeal process mentioned previously by demonstrating that the pieces can be composed into a coherent, verified whole.

Preliminary Recommendations

The following preliminary recommendations are oriented toward frontier AI labs and those responsible for developing and maintaining LLM inference and training stacks, which is this report’s primary audience. These recommendations reflect our synthesis of the survey findings. We worked under the assumption that the timelines that we described in Chapter 1 are plausible and that stakeholders must prepare for the outcomes. The recommendations are intended as a starting point for discussion among the relevant communities; they are not settled guidance.

Many of the underlying verification approaches, such as cryptographic boundary enforcement, access-control specification, and sandbox verification, apply across ML deployment contexts, but the prioritization would differ substantially for other domains. Safety-critical embedded systems (e.g., autonomous vehicles, medical devices) face different constraint profiles: tighter real-time requirements, different hardware targets, and regulatory regimes that already mandate certain assurance levels. A companion analysis that maps verification priorities to these other ML contexts would be a valuable extension of this work.

For Frontier AI Labs and Hyperscalers

Verified cryptographic libraries are the clearest near-term deployment target that experts identified and are likely to offer immediate value for some weight-protection use cases—as discussed previously, the HACLS/EverCrypt family of libraries (Zinzindohoué et al., 2017; Protzenko et al., 2020) is production-ready and already deployed widely (Project Everest, undated; Microsoft Research, 2021).

We recommend that stakeholders

- develop formal specifications for access control and safety monitor requirements (access control was among the highest-rated components for both security value and verification feasibility; safety monitor chains received more mixed assessments, and respondents broadly supported verification that monitors cannot be bypassed while also questioning the value of verifying monitor effectiveness [see Chapter 3])
- architect systems to separate verified invariant-enforcing components from unverified performance-optimized code—which was the architectural pattern that respondents most consistently endorsed—thus enabling verification without sacrificing development velocity.

For the Formal Methods Community

Stakeholders should

- engage with AI safety requirements to understand what properties matter most for ML infrastructure security

- develop benchmarks for AI-assisted systems verification, particularly benchmarks targeting proof repair and systems-level reasoning about memory, state, and concurrency
- develop verification approaches that accommodate rapidly evolving codebases by separating verified security-enforcement components from unverified performance-optimized code, putting formally specified interfaces between them, thus enabling verification without impeding development velocity.

For Hardware Vendors

Stakeholders should publish formal semantics or verifiable interfaces for hardware instruction set architectures and memory models. The lack of mature, public formal semantics or verifiable interface specifications for modern accelerators is the most acute constraint on ML infrastructure verification depth. However, CPUs and memory subsystems similarly lack adequate formal specifications, which limits verification across the entire hardware-software boundary. Broader engagement from hardware vendors would unlock the verification of components across the full computing stack, not only the ML-specific layers.

For Government Agencies

Stakeholders should

- fund comprehensive formal methods benchmarks within three to 12 months of this report’s publication (Sarma and Fisher, 2026), building on DARPA’s Information Innovation Office’s existing formal methods portfolio, because creating such benchmarks is a key enabler for developing the AI capabilities that make infrastructure verification economically viable
- consider procurement requirements for verified components in high-assurance AI applications
- support workforce development in formal methods; Chinese labs have made notable advances in AI-assisted theorem proving (Ren et al., 2025), which warrants a competitive response.

For the Broader Community

Academic researchers, open-source maintainers, and independent security practitioners can participate in developing shared, open-source verified components (e.g., cryptographic libraries, protocol implementations, parser generators) that reduce duplication of efforts. They can also contribute to specification libraries and red-teaming efforts that improve specification quality—the bottleneck that AI assistance is least equipped to address. A particularly high-value contribution would be developing an open-source, minimally verified inference stack—a verified analogue of Karpathy’s `llm.c` (Karpathy, 2025)—which would demonstrate that end-to-end verified inference is achievable at a modest scale, thus providing a concrete foundation that frontier labs could use for production purposes. Early efforts to formalize neural network semantics directly within theorem provers (George et al., 2026) represent a complementary approach, closing the semantic gap between executed models and verified artifacts.

Conclusion

The findings from our survey suggest that verified ML infrastructure is both feasible and necessary. It is feasible because the foundational techniques—such as verified OS kernels, verified cryptography, and verified distributed protocols—exist, although these efforts have not yet been widely adopted into production ML systems, and significant engineering and research challenges remain to be solved before feasibility is demonstrated at scale. It is necessary because the security of AI infrastructure is becoming synonymous with AI security: A model that can exploit its own infrastructure is a model that cannot be reliably controlled. Key research challenges for accomplishing these aims include developing formal semantics for hardware accelerators, specification methodologies for concurrent systems, verification of hyperproperties, and formal treatment of the interfaces between verified and unverified components.

In the coming years, there will be rapid improvements in AI capabilities, including capabilities directly relevant to formal verification. By investing now in specifications, benchmarks, and coordination mechanisms, the United States can position itself to leverage these capabilities as they emerge. The goal is not to verify everything today but to build the infrastructure and community that can verify what matters as capabilities mature. Given the pace of change, we recommend that stakeholders convene regularly to reassess priorities as the field evolves and capture those findings in a document. A document, such as this report, is best understood not as a fixed plan but as a snapshot of expert judgment at a moment when underlying capabilities are advancing rapidly.

Our findings point toward pragmatic preliminary steps: Start with cryptographic boundaries (verified implementations of key libraries already exist), expand to safety-critical control logic as AI-assisted tools improve, and maintain honest boundaries around what verification can and cannot achieve. The collection of experts who contributed to this effort—formal methods researchers, AI safety practitioners, infrastructure engineers, and policymakers—represents the kind of cross-disciplinary community that is needed to execute this agenda. The most urgent next step is to convene that community and begin refining the priorities identified here, ideally within weeks rather than months.

Detailed Methodology

Phased Approach

We employed LLM-generated simulated expert responses as a development tool for the survey instrument and synthesis framework. We followed this initial step by holding a structured consultation with human experts of various domains. In the development phase, simulated responses generated from LLM-constructed personas helped refine our question framing, identify gaps in topical coverage, and produce preliminary synthesis documents that informed the structure of this report. All findings in this report derive from the survey of human experts that we conducted.

LLM-Assisted Survey Development

Prior to administering the survey, we generated simulated responses using LLM-constructed expert personas to stress-test the instrument. Each persona specification included their primary domain of expertise, secondary areas of knowledge, years of experience, institutional context, sector-specific experience, and relevant methodological orientations. These personas were designed not to represent specific real individuals but to instantiate archetypal expert profiles spanning a variety of relevant expertise. After analyzing the simulated responses, we identified ambiguities in question framing, gaps in topical coverage, and areas in which the survey instrument could likely elicit more-precise responses. The review of the simulated responses also helped us produce preliminary synthesis documents that served as scaffolding for this report's structure. This approach has broader potential applications: In domains in which expertise is highly specialized, proprietary, or geographically concentrated, LLM-simulated expert responses could serve as a tool for building preliminary consensus frameworks and identifying key areas of agreement and disagreement before experts commit their limited time. Future work in this research program will explore this use case more fully.

Human Expert Survey Administration

Following approval from RAND's Human Subjects Protection Committee, we distributed surveys to domain experts whom we selected from our professional networks and through targeted outreach. Respondents completed the survey instruments independently. Respondents completed questions about their backgrounds, which enabled us to contextually group respondents by their domains, sectors, and experience levels without individual attribution.

Synthesis

We synthesized the responses of the human experts using a multistage process combining LLM-assisted analysis with independent human validation. In the first stage, open-ended responses were systematically summarized and coded using LLM-assisted qualitative analysis methods: We used inductive coding to identify emergent themes and cross-case comparisons to surface patterns of agreement and divergence, and we integrated technical details into coherent narrative summaries. In the second stage, a team member with expertise in qualitative research design and analysis independently reviewed the open-ended survey responses and identified key themes. This independent validation served as a methodological check on the LLM-assisted synthesis, ensuring that the themes, patterns, and findings reported in the analysis faithfully represented respondent input rather than reflecting artifacts of the LLM summarization process. We compared the themes we identified through the independent qualitative review against the LLM-assisted analysis; any convergence provided additional confidence in the findings’ robustness. When the independent review surfaced themes or nuances that the LLM-assisted synthesis did not capture, we incorporated those themes or nuances into the report analysis and findings. The LLM-generated preliminary synthesis from the development phase informed the report’s organizational structure, but we did not take findings, ratings, or quotations from the synthesis. When the preliminary synthesis identified considerations that human respondents also raised, that convergence gave us additional confidence in the relevance of those themes. When human respondents raised considerations that were not in the preliminary synthesis, we treated these themes as particularly valuable additions reflecting the experts’ tacit or experiential knowledge.

Distinguishing This Approach from Silicon Sampling

It is essential to distinguish the LLM-assisted development approach used here from *silicon sampling*, which is the use of LLMs to simulate survey responses or estimate population parameters (Argyle et al., 2023). We used LLM-generated content solely as a development and structuring tool; we did not treat simulated responses as data or draw findings from them. The value of the LLM’s contribution was its capacity to identify relevant considerations, articulate arguments that experts might recognize as legitimate, and structure the problem space in ways that facilitated subsequent human deliberation. A more ambitious use of LLM-simulated expert responses—such as by using them as a tool for building preliminary technical consensus across highly specialized communities—is a promising direction for future work. In such domains as formal methods, hardware verification, and AI safety—domains in which relevant expertise is scarce, geographically dispersed, and often bound by proprietary constraints—simulated expert consultation could help develop shared roadmaps and surface areas of likely agreement before convening expensive in-person workshops. A companion report in this research program will explore this approach.

Limitations

The sample of human experts represents a convenience sample rather than a probability sample. Findings should be interpreted as reflecting the perspectives of consulted experts rather than as consensus views within broader professional communities. The sample draws most heavily from research and academia, although a majority of respondents reported practitioner or hybrid backgrounds; additional input from frontier lab practitioners and hardware-side experts would strengthen the findings. The LLM-assisted survey-development process, while useful for refining the instrument, might have imposed structural biases on question framing that shaped the response space. Additionally, the predefined risk categories in the

survey instrument conflated attack objectives (e.g., weight exfiltration) with attack vectors (e.g., remote code execution via framework vulnerabilities). Future iterations of the instrument (or a similar one) should separate these dimensions to enable more precise risk analysis and clearer mapping between threats and verification strategies.

Survey Instrument Summary

The full survey comprised ten sections covering the following areas:

- **Section 1: Purpose and Context.** Respondents were provided with an overview of the study’s purpose, a definition of *verified ML infrastructure stack*, and a rationale for the timeliness of the work.
- **Section 2: Informed Consent for Participation in Research: Security and Addressing Critical Infrastructure Challenges of Emerging AI Survey.** Respondents completed a standard RAND informed-consent statement before proceeding.
- **Section 3: Security and Safety Risks.** Respondents were presented with 12 predefined risk categories and asked to (1) add any missing risks, (2) rate each risk’s severity (as low, medium, high, or critical), (3) assess whether each risk could be meaningfully mitigated through formal verification (available responses were “yes, completely,” “yes, partially,” “no” or “don’t know”), and (4) explain their verifiability assessments.
- **Section 4: Feasibility and Adoption Barriers.** This section included open-ended questions on technical barriers, adoption barriers, stakeholder requirements, and the patch velocity tension.
- **Section 5: Component Priorities.** Respondents rated 15 ML infrastructure components on their security value (using a five-point scale) and verification feasibility (using a similar five-point scale); there was also the option to provide open-ended explanations.
- **Section 6: Specifications.** This section included questions on target properties, the value of incomplete specifications, and specification-governance processes.
- **Section 7: AI-Assisted Formal Methods.** Respondents were asked to give structured ratings of the expected impact that AI might have on proof synthesis, proof repair, specification elicitation, and tool and protocol synthesis; respondents could give explanations and identify bottlenecks.
- **Section 8: Related Work and Resources; Section 9: Additional Considerations; and Section 10: Background (for Contextual Grouping Only).** These sections included space for respondents to detail their related work and resources, additional considerations, and background demographics.

The complete survey instrument is available in an annex to this report.

Detailed Survey Data

Tables C.1 through C.4 show the results from our surveys.

TABLE C.1
Risk Severity Ratings

Risk Category	Mean	Critical	High	Medium	Low	Don't Know
Tampering with stored weights	3.29	11	7	1	2	0
Supply chain compromise	3.19	7	12	1	1	0
Weight exfiltration	3.15	11	4	2	3	1
Remote code execution	3.10	7	9	3	1	1
Bypass of safety monitors	2.81	6	6	8	1	0
Training data poisoning	2.75	3	10	6	1	1
Model manipulation of evaluations	2.75	3	10	6	1	1
Checkpoint integrity	2.75	4	8	7	1	1
Unsafe deserialization	2.56	4	5	3	4	5
Inadequate isolation	2.55	3	7	8	2	1
Side-channel leakage	1.95	1	3	9	6	2
Distributed inconsistencies	1.78	0	3	8	7	3

NOTE: Respondents ($n = 21$) rated each risk on a four-point scale: low (1), medium (2), high (3), and critical (4). "don't know" responses were excluded from mean calculations. Other than the "Mean" column, the table shows the number of responses.

TABLE C.2
Verifiability by Formal Methods

Risk Category	Yes, Completely	Yes, Partially	No	Don't Know
Remote code execution	10	10	1	1
Inadequate isolation	10	9	1	2
Unsafe deserialization	11	6	0	5
Distributed inconsistencies ^a	7	12	0	2
Tampering with stored weights	6	14	2	0
Supply chain compromise	6	10	3	3
Checkpoint integrity	5	13	1	3
Bypass of safety monitors	5	11	4	2
Weight exfiltration	3	14	3	2
Model manipulation of evaluations	3	9	7	3
Training data poisoning	2	7	10	3
Side-channel leakage	1	9	9	3

NOTE: Respondents ($n = 22$) assessed whether each risk could be meaningfully mitigated through formal verification. The table shows the number of responses for each category.

^a $n = 21$. One respondent did not provide a response for this risk category.

TABLE C.3
Component Verification Priorities

Component	Security Value	Feasibility	Priority
Cryptographic primitives	3.83	4.47	17.12
Access control and authentication	3.89	4.16	16.18
Sandbox/isolation boundaries	3.89	3.17	12.33
OS/hypervisor	3.78	2.68	10.13
Communication protocols (TLS, RPC)	3.59	3.72	13.35
Hardware drivers	3.33	2.47	8.23
GPU kernels and CUDA code	3.00	2.22	6.66
ML framework core	2.94	2.06	6.06
KV cache transfer/storage	3.06	—	—
Safety monitor chain	3.00	3.17	9.51
Model loading and deserialization	2.94	3.74	11.00
Logging and audit infrastructure	2.82	—	—
Training orchestration	2.78	3.06	8.51
Distributed coordination	2.17	2.94	6.38
Request routing and load balancing	2.00	3.28	6.56

NOTE: The table shows the mean scores. Security value was rated on a five-point scale (1 = minimal, addresses less than 10 percent of the relevant attack surface; 2 = low, addresses 11 to 25 percent of the relevant attack surface; 3 = moderate, addresses 26–50 percent of the relevant attack surface or is a meaningful layer in defense-in-depth; 4 = high, addresses 51–75 percent of the relevant attack surface or is critical to multiple threat categories; 5 = very high, greater than 75 percent of the relevant attack surface or is a foundational component, and compromise here undermines the broader security posture). Feasibility was rated on a five-point scale with only the endpoints labeled (1 = very difficult; 5 = feasible). We calculated the priority score by multiplying the security value score by the feasibility score. This simple ranking heuristic is used to order components by combined importance and tractability; it is not intended as a precise composite measure. A dash (—) indicates that there were insufficient responses to calculate a reliable mean.

TABLE C.4
Expected Impact of AI-Assisted Formal Methods in Two to Five Years

Method	Feasibility					Cost				
	Sig+	Some+	None	Harder	DK+/-	Sig↓	Some↓	None	Higher	DK\$
Proof repair	17	4	0	0	1	17	5	0	0	0
Proof synthesis	16	5	0	0	1	16	6	0	0	0
Tool and protocol synthesis	11	8	1	0	2	13	7	0	0	2
Specification elicitation	8	7	5	1	1	7	9	4	1	1

NOTE: The table shows the number of responses ($n = 22$). Sig+ = significantly easier; Some+ = somewhat easier; DK+/- = don't know, easier or harder; Sig↓ = significantly lower cost; Some↓ = somewhat lower cost; DK\$ = don't know, cost higher or lower.

Respondent Profile

Twenty-three experts completed the survey; 22 provided background information. Tables D.1 through D.3 summarize respondent demographics for those 22 respondents.

TABLE D.1
Primary Domain

Domain	<i>n</i>	%
Research and academia	10	45
Both or hybrid	7	32
Practitioner and engineering	5	23

TABLE D.2
Years of Experience

Range	<i>n</i>	%
11–20 years	12	55
6–10 years	7	32
0–5 years	3	14

NOTE: The percentages do not add up to 100 exactly because of rounding.

TABLE D.3
Areas of Expertise (Multiple Selections Permitted)

Area	<i>n</i>	%
Software engineering	11	50
Formal methods and verification	11	50
AI infrastructure and ML systems	11	50
Cybersecurity	8	36
Policy and governance	4	18
Hardware and systems architecture	1	5
Other (AI safety, ML, futurism, materials)	4	18

Abbreviations

AI	artificial intelligence
AWS	Amazon Web Services
CPU	central processing unit
DARPA	Defense Advanced Research Projects Agency
GPU	graphics processing unit
LLM	large language model
ML	machine learning
OS	operating system
RPC	remote procedure call
syscall	system call
TLS	Transport Layer Security

References

- Abraham, Lisa, Joshua Kavner, and Alvin Moon, *A Prisoner's Dilemma in the Race to Artificial General Intelligence*, RAND Corporation, RR-A4245-1, 2026. As of May 5, 2026:
https://www.rand.org/pubs/research_reports/RR4245-1.html
- American Hospital Association, "AHA Survey: Change Healthcare Cyberattack Having Significant Disruptions on Patient Care, Hospitals' Finances," March 15, 2024.
- Anthropic, "Project Glasswing: Securing Critical Software for the AI Era," webpage, undated. As of May 26, 2026:
<https://www.anthropic.com/glasswing>
- Argyle, Lisa P., Ethan C. Busby, Nancy Fulda, Joshua R. Gubler, Christopher Rytting, and David Wingate, "Out of One, Many: Using Language Models to Simulate Human Samples," *Political Analysis*, Vol. 31, No. 3, July 2023.
- Avigad, Jeremy, "Mathematicians in the Age of AI," arXiv, arXiv:2603.03684v3, April 6, 2026.
- Betts, Adam, Nathan Chong, Alastair F. Donaldson, Jeroen Ketema, Shaz Qadeer, Paul Thomson, and John Wickerson, "The Design and Implementation of a Verification Technique for GPU Kernels," *ACM Transactions on Programming Languages and Systems*, Vol. 37, No. 3, May 2015.
- Blanchet, Bruno, "An Efficient Cryptographic Protocol Verifier Based on Prolog Rules," *14th IEEE Computer Security Foundations Workshop*, June 2001.
- Boudreaux, Benjamin, Michael J. D. Vermeer, Kamaria Horton, and Nidhi Kalra, *The Case for AI Loss of Control Response Planning and an Outline to Get Started*, RAND Corporation, PE-A4232-1, October 2025. As of May 26, 2026:
<https://www.rand.org/pubs/perspectives/PEA4232-1.html>
- Bursuc, Sergiu, Theodore Ehrenborg, Shaowei Lin, Lacramioara Astefanoaei, Ionel Emilian Chiosa, Jure Kukovec, Alok Singh, Oliver Butterley, Adem Bizid, Quinn Dougherty, et al., "A Benchmark for Vericoding: Formally Verified Program Synthesis," arXiv, arXiv:2509.22908, September 26, 2025.
- Cedar, homepage, undated. As of April 14, 2026:
<https://www.cedarpolicy.com/en>
- CERT Coordination Center, "SGLang (sglang) Is Vulnerable to Code Execution Attacks Via Unsafe Pickle Deserialization," Carnegie Mellon University, Vulnerability Note VU#665416, last updated April 7, 2026.
- Chatterjee, Bodhisatwa, Drew Zagieboylo, Sana Damani, Siva Hari, and Christos Kozyrakis, "ProofWright: Towards Agentic Formal Verification of CUDA," arXiv, arXiv:2511.12294v1, November 15, 2025.
- Cybersecurity and Infrastructure Security Agency, "Apache Log4j Vulnerability Guidance," U.S. Department of Homeland Security, April 8, 2022.
- Cook, Byron, "An Unexpected Discovery: Automated Reasoning Often Makes Systems More Efficient and Easier to Maintain," *AWS Security Blog*, October 17, 2024. As of May 26, 2026:
<https://aws.amazon.com/blogs/security/an-unexpected-discovery-automated-reasoning-often-makes-systems-more-efficient-and-easier-to-maintain/>
- CVE-2025-62164, "vLLM Deserialization Vulnerability Leading to DoS and Potential RCE," GitHub Advisory Database, November 20, 2025. As of May 5, 2026:
<https://github.com/advisories/GHSA-mrw7-hf4f-83pf>
- CVE-2025-9141, "vLLM Has Remote Code Execution Vulnerability in the Tool Call Parser for Qwen3-Coder," GitHub Advisory Database, August 21, 2025. As of May 5, 2026:
<https://github.com/advisories/GHSA-79j6-g2m3-jgfw>
- DARPA—See Defense Advanced Research Projects Agency.
- Defense Advanced Research Projects Agency, "expMath: Exponentiating Mathematics," webpage, undated-a. As of June 25, 2025:
<https://www.darpa.mil/research/programs/expmath-exponential-mathematics>

- Defense Advanced Research Projects Agency, “Formal Methods,” webpage, undated-b. As of April 14, 2026: <https://www.darpa.mil/research/research-spotlights/formal-methods>
- Freund, Andres, “Backdoor in Upstream xz/liblzma Leading to SSH Server Compromise,” email to Openwall, March 29, 2024. As of May 5, 2026: <https://www.openwall.com/lists/oss-security/2024/03/29/4>
- George, Robert Joseph, Jennifer Cruden, Xiangru Zhong, Huan Zhang, and Anima Anandkumar, “TorchLean: Formalizing Neural Networks in Lean,” arXiv, arXiv:2602.22631, February 26, 2026.
- Google DeepMind, “AI Achieves Silver-Medal Standard Solving International Mathematical Olympiad Problems,” blog post, July 25, 2024. As of May 5, 2026: <https://deepmind.google/discover/blog/ai-solves-imo-problems-at-silver-medal-level/>
- Hathhorn, Chris, Michela Becchi, William L. Harrison, and Adam Procter, “Formal Semantics of Heterogeneous CUDA-C: A Modular Approach with Applications,” arXiv, arXiv:1211.6193, November 27, 2012.
- Hawblitzel, Chris, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill, “IronFleet: Proving Practical Distributed Systems Correct,” *Proceedings of the 25th Symposium on Operating Systems Principles*, October 2015.
- Karpathy, Andrej, “llama2.c,” source code, GitHub, last updated May 29, 2024. As of May 5, 2026: <https://github.com/karpathy/llama2.c>
- Karpathy, Andrej, “llm.c,” source code, GitHub, last updated May 10, 2025. As of May 5, 2026: <https://github.com/karpathy/llm.c>
- Klein, Gerwin, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, et al., “seL4: Formal Verification of an OS Kernel,” *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, October 2009.
- Kwa, Thomas, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, et al., “Measuring AI Ability to Complete Long Tasks,” blog post, METR, March 19, 2025. As of April 14, 2026: <https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/>
- Leroy, Xavier, “Formal Verification of a Realistic Compiler,” *Communications of the ACM*, Vol. 52, No. 7, July 2009.
- LLVM Project, “Compiling CUDA with Clang,” webpage, undated. As of April 10, 2026: <https://llvm.org/docs/CompileCudaWithLLVM.html>
- Lustig, Daniel, Sameer Sahasrabuddhe, and Olivier Giroux, “A Formal Analysis of the NVIDIA PTX Memory Consistency Model,” *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, April 2019.
- Math, Inc., “Completing the Formal Proof of Higher-Dimensional Sphere Packing,” webpage, undated-a. As of May 5, 2026: <https://www.math.inc/sphere-packing>
- Math, Inc., “Introducing Gauss, an Agent for Autoformalization,” webpage, undated-b. As of May 5, 2026: <https://www.math.inc/gauss>
- Meier, Simon, Benedikt Schmidt, Cas Cremers, and David Basin, “The TAMARIN Prover for the Symbolic Analysis of Security Protocols,” *Proceedings of the 25th International Conference on Computer Aided Verification*, Vol. 8044, July 2013.
- Microsoft Research, “Creating the Future of Software Development,” blog post, May 26, 2021. As of May 5, 2026: <https://www.microsoft.com/en-us/research/blog/research-collection-creating-the-future-of-software-development/>
- Miranda, Brando, Zhanke Zhou, Allen Nie, Elyas Obbad, Leni Aniva, Kai Fronsda, Weston Kirk, Dilara Soyulu, Andrea Yu, Ying Li, et al., “VeriBench: End-to-End Formal Verification Benchmark for AI Code Generation in Lean 4,” *Proceedings of the 42nd International Conference on Machine Learning*, July 2025.
- Mitch, Ian, Matthew J. Malone, Karen Schwindt, Gregory Smith, Wesley Hurd, Henry Alexander Bradley, and James Gimbi, *Governance Approaches to Securing Frontier AI*, RAND Corporation, RR-A4159-1, 2025. As of May 5, 2026: https://www.rand.org/pubs/research_reports/RRA4159-1.html

- Nevo, Sella, Dan Lahav, Ajay Karpur, Yogev Bar-On, Henry Alexander Bradley, and Jeff Alstott, *Securing AI Model Weights: Preventing Theft and Misuse of Frontier Models*, RAND Corporation, RR-A2849-1, 2024. As of May 5, 2026:
https://www.rand.org/pubs/research_reports/RRA2849-1.html
- Nolan, Beatrice, “Exclusive: Anthropic Acknowledges Testing New AI Model Representing ‘Step Change’ in Capabilities, After Accidental Data Leak Reveals Its Existence,” *Fortune*, March 26, 2026.
- NVIDIA, “Security Bulletin: NVIDIA TensorRT LLM—April 2025,” webpage, last updated May 1, 2025. As of May 5, 2026:
https://nvidia.custhelp.com/app/answers/detail/a_id/5648/~security-bulletin%3A-nvidia-tensorrt-llm---april-2025
- Parfomak, Paul W., and Chris Jaikaran, “Colonial Pipeline: The DarkSide Strikes,” Congressional Research Service, IN11667v2, May 11, 2021.
- Pavel, Barry, Ivana Ke, Gregory Smith, Sophia Brown-Heidenreich, Lea Sabbag, Ashwin Acharya, and Yusuf Mahmood, *How Artificial General Intelligence Could Affect the Rise and Fall of Nations: Visions for Potential AGI Futures*, RAND Corporation, RR-A3034-2, 2025. As of May 5, 2026:
https://www.rand.org/pubs/research_reports/RRA3034-2.html
- Project Everest, homepage, undated. As of April 10, 2026:
<https://project-everest.github.io/>
- Protzenko, Jonathan, Bryan Parno, Aymeric Fromherz, Chris Hawblitzel, Marina Polubelova, Karthikeyan Bhargavan, Benjamin Beurdouche, Joonwon Choi, Antoine Delignat-Lavaud, Cédric Fournet, et al., “EverCrypt: A Fast, Verified, Cross-Platform Cryptographic Provider,” *2020 IEEE Symposium on Security and Privacy (SP)*, May 2020.
- Ren, Z. Z., Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, et al., “DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition,” arXiv, arXiv:2504.21801v2, July 18, 2025.
- Saraswat, Vishesh, Jeffrey Chang, Harshikaa Agrawal, Vaidehi Agarwalla, Robert Zhang, Twm Stone, Jacob Green, Shiki Vaahan, Lawrence Chan, Rajashree Agrawal, et al., “lf-lean: The Frontier of Verified Software Engineering,” *Theorem Blog*, February 5, 2026. As of May 5, 2026:
<https://theorem.dev/blog/lf-lean/>
- Sarma, Gopal P., and Kathleen Fisher, “Tipping the Cyber Balance: How AI Benchmarks Could Make Software Safer,” commentary, RAND Corporation, February 3, 2026. As of May 5, 2026:
<https://www.rand.org/pubs/commentary/2026/02/tipping-the-cyber-balance-how-ai-benchmarks-could-make.html>
- Sarma, Gopal P., Sunny D. Bhatt, Michael Jacob, and Rachel Steratore, *Artificial General Intelligence Forecasting and Scenario Analysis: State of the Field, Methodological Gaps, and Strategic Implications*, RAND Corporation, RR-A4692-1, 2026. As of May 26, 2026:
https://www.rand.org/pubs/research_reports/RRA4692-1.html
- seL4, “seL4 Proofs,” webpage, undated. As of May 5, 2026:
<https://sel4.systems/Verification/proofs.html>
- seL4, “seL4 Version 14.0.0 Release,” webpage, November 25, 2025. As of May 5, 2026:
<https://docs.sel4.systems/releases/sel4/14.0.0.html>
- Wilcox, James R., Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas Anderson, “Verdi: A Framework for Implementing and Formally Verifying Distributed Systems,” *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, June 2015.
- Yang, Kaiyu, Gabriel Poesia, Jingxuan He, Wenda Li, Kristin Lauter, Swarat Chaudhuri, and Dawn Song, “Formal Mathematical Reasoning—A New Frontier in AI,” *Proceedings of the 42nd International Conference on Machine Learning*, July 2025.
- Ye, Zhe, Zhengxu Yan, Jingxuan He, Timothe Kasriel, Kaiyu Yang, and Dawn Song, “VERINA: Benchmarking Verifiable Code Generation,” arXiv, arXiv:2505.23135v2, October 18, 2025.
- Zinzindohoué, Jean Karim, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche, “HACL*: A Verified Modern Cryptographic Library,” *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, October 2017.

About the Authors

Gopal P. Sarma is a strategic technology leader and a member of the artificial intelligence (AI) security cluster at RAND. He conducts research on policy developments and portfolio management related to emerging technologies and national security. He holds an M.D. and a Ph.D. in applied physics and has extensive experience in entrepreneurship and science policy.

Rachel Steratore is a sociotechnical researcher at RAND. She conducts technical and policy research on how people interpret, evaluate, and trust AI and other emerging technologies, with the goal of supporting transparent, safe, and equitable innovation. Her work integrates cognitive research, systems thinking, and governance strategies to inform national programs and institutional decisionmaking on AI evaluation, risk assessment, and technology policy. She holds a Ph.D. in engineering and public policy.

Sunny D. Bhatt is a senior policy analyst at RAND. He conducts research on national security, defense, and intelligence issues, including chemical, biological, radiological, nuclear, and high-yield explosives; organizational performance; and AI. He holds an M.A. in public policy.

Geoffrey Irving is the chief scientist at the Artificial Intelligence Security Institute in the United Kingdom. His research focus spans AI safety, alignment, and neural network theorem proving. Previously, he led the Scalable Alignment Team at Google DeepMind and the Reflection Team at OpenAI, and he conducted research on deep learning-guided proof search at Google Brain. He holds a Ph.D. in computer science.